

Магистратура
«Искусственный интеллект»

Инструменты разработки
Лекция 7

Чернов Евгений
НИУ ВШЭ

Docker и devcontainers

Как описать образ в Dockerfile, запускать контейнеры с портами и данными и сделать среду разработки воспроизводимой

План занятия

1. **От ядра к Docker**
   что знаем из лекции 6 и что добавляет Docker
2. **Архитектура**
   клиент, демон, реестр и имя образа
3. **Dockerfile**
   инструкции, кэш слоёв, CMD и ENTRYPOINT
4. **Запуск и данные**
   docker run, жизненный цикл, тома
5. **Сеть**
   публикация портов и контейнеры по имени
6. **Качество и среда**
   маленький образ, не root, devcontainers

От ядра к Docker

В прошлый раз мы разобрали контейнер руками: namespaces, cgroups, слои образа. Docker не добавляет новой изоляции — он упаковывает приложение, хранит образы и запускает их одной командой.

Уже знаем (лекция 6)

- контейнер — процесс на общем ядре
- namespaces — что видно, cgroups — сколько можно
- образ — слои только для чтения
- контейнер = образ + тонкий слой для записи

Сегодня

- как описать образ — Dockerfile
- как запускать и где хранить данные
- как связать контейнеры сетью
- как сделать образ маленьким и безопасным

! **Сегодня — инструмент, а не ядро:** команды и практики, с которыми вы будете работать каждый день.

01

РАЗДЕЛ 01

Как устроен Docker

Клиент и демон, реестр и имя образа — кто что делает, когда вы пишете docker run

Клиент, демон и реестр

Docker — клиент-серверная система. Команда docker — только клиент: она отправляет запрос демону dockerd, а уже демон собирает образы, запускает контейнеры и ходит в реестр.



`docker build · run · ps`
команды клиента; сам клиент ничего не запускает

`/var/run/docker.sock`
Unix-сокет: по нему клиент говорит с демоном

`pull · push`
демон скачивает образы из реестра и отправляет их туда

Имя образа: где лежит, чей и какой версии

Образ находят по имени. Полное имя говорит, в каком он реестре, чей он и какая версия. Короткие имена — сокращения, Docker дописывает остальное сам.

`ghcr.io` / `aesedeu` / `hello` `:1.2`

реестр | владелец | репозиторий | тег (версия)

Как пишут	Как понимает Docker	Комментарий
python:3.12-slim	docker.io/library/python:3.12-slim	официальный образ
nginx	docker.io/library/nginx:latest	latest — тег по умолчанию
nginx@sha256:3f1a...	точный слепок по хешу	не меняется никогда

02

РАЗДЕЛ 02

Dockerfile

Рецепт образа: инструкции по шагам, кэш слоёв и то, что выполнится при старте

Dockerfile: образ описан по шагам

Dockerfile

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt
COPY . .
ENV PORT=8000
EXPOSE 8000
CMD ["python", "app.py"]
```

FROM базовый образ, с которого начинаем

WORKDIR рабочая папка внутри образа

COPY положить файлы проекта в образ

RUN выполнить команду при сборке

ENV · **ARG** переменные запуска · сборки

EXPOSE документирует порт, не открывает его

CMD что запустить при старте контейнера

Собираем Dockerfile для FastAPI

Разберём боевой Dockerfile маленького FastAPI-сервиса — строка за строкой.

Dockerfile · FastAPI

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app",
     "--host", "0.0.0.0", "--port", "8000"]
```

1. **FROM …slim** — лёгкий Python как база
2. **WORKDIR /app** — папка внутри образа
3. **COPY requirements** — сначала только зависимости
4. **RUN pip install** — ставим, слой кэшируется
5. **COPY . .** — теперь код приложения
6. **--host 0.0.0.0** — слушать все интерфейсы

Кэш слоёв экономит время пересборки

Docker кэширует каждый шаг. Изменился шаг — пересобираются он и все шаги ниже. Поэтому порядок инструкций в Dockerfile — это стратегия кэша.

плохо — код раньше зависимостей

```
FROM python:3.12-slim
COPY . .
RUN pip install -r requirements.txt

правка кода → pip заново, минуты
```

хорошо — зависимости раньше

```
FROM python:3.12-slim
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
# правка кода → pip из кэша, секунды
```

Правило: что меняется редко (система, зависимости) — выше; что меняется часто (код) — ниже.

CMD и ENTRYPOINT: что выполнится при старте

Обе инструкции задают команду запуска, но по-разному обходятся с аргументами docker run.

CMD — команда по умолчанию

```
CMD ["python", "app.py"]

$ docker run app
# → python app.py
$ docker run app bash
→ bash   (CMD заменён целиком)
```

ENTRYPOINT + CMD

```
ENTRYPOINT ["python", "app.py"]
CMD ["--port", "8000"]
$ docker run app
# → python app.py --port 8000
$ docker run app --port 80
# → python app.py --port 80
```

Пишите в exec-форме — JSON-массивом. Строка CMD python app.py запускается через /bin/sh: главным процессом становится оболочка, сигнал остановки получает она, и приложение не успевает корректно завершиться.

ARG и ENV: переменные сборки и запуска

	ARG	ENV
Когда доступна	только при сборке (docker build)	при сборке и в работающем контейнере
Как задать	--build-arg PY=3.12	ENV в Dockerfile или -e при docker run
Пример	версия базы, флаги сборки	порт, режим работы, адрес БД
Секреты?	нет — видны в docker history	нет — видны в docker inspect

Пароли и токены не кладут ни в ARG, ни в ENV, ни через COPY .env — они остаются в слоях и метаданных образа. Передавайте их при запуске.

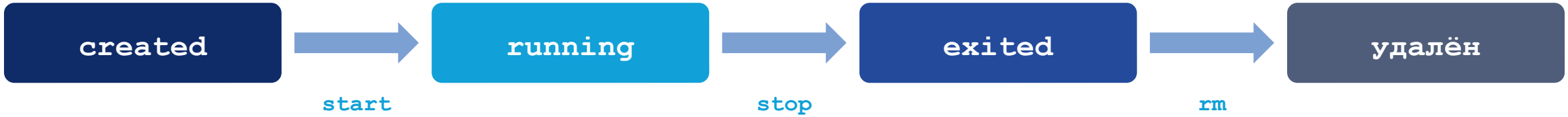
03

РАЗДЕЛ 03

Запуск, данные и сеть

Жизненный цикл контейнера, где хранить данные и как контейнеры находят друг друга

docker run и жизненный цикл контейнера



терминал

```
# run = create + start
$ docker run -d --name web \
    -p 8080:8000 -e WHO=HSE hello:1
$ docker ps
web   Up 3 seconds   0.0.0.0:8080->8000/tcp
$ docker stop web && docker rm web
```

-d в фоне
--name своё имя
-p порт хост:контейнер
-e переменная окружения
-v том или папка
--rm удалить после выхода

Volumes и bind mounts: где живут данные

Слой для записи исчезает вместе с контейнером. Всё, что должно пережить docker rm, монтируют снаружи.

Named volume

```
-v pgdata:/var/lib/postgresql/data
```

- данные хранит и управляет Docker
- переживает docker rm контейнера
- для баз данных и состояния

Bind mount

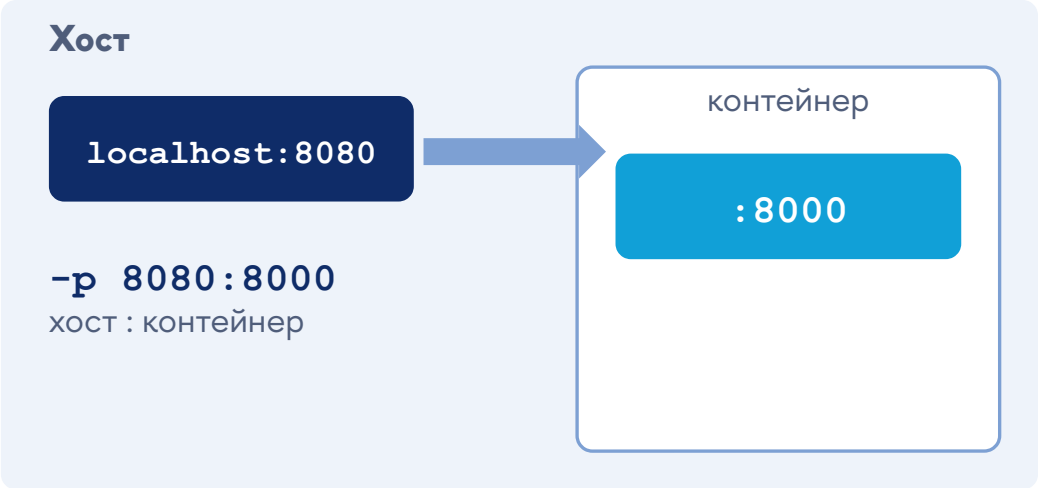
```
-v $PWD:/app
```

- папка хоста видна внутри контейнера
- правите код на хосте — сразу в контейнере
- для разработки и hot-reload

Именованный том удаляется только явно: `docker volume rm pgdata`. Список томов — `docker volume ls`.

Сеть: публикация портов и контейнеры по имени

У контейнера своя сеть — net namespace из лекции 6. Снаружи до него не достучаться, пока порт не опубликован флагом -p.



контейнеры в одной сети

```
$ docker network create appnet
$ docker run -d --name db --network appnet \
    -e POSTGRES_PASSWORD=pass postgres:16
$ docker run -d --name api --network appnet app
# из api: postgres://db:5432 — имя = DNS
```

Слушайте 0.0.0.0, а не 127.0.0.1: 127.0.0.1 внутри контейнера — это сам контейнер, и -p до приложения не достучится.

Типы сетей Docker: какую когда брать

Сеть	Что это	Когда использовать
bridge по умолчанию	сеть docker0, сюда попадает контейнер без --network; по именам друг друга не видят — только по IP	разовый запуск одного контейнера
своя bridge	docker network create appnet; встроенный DNS — контейнеры находят друг друга по имени	приложение + БД на одной машине — выбор по умолчанию
host	своей сети нет: процесс слушает порты прямо на хосте, -p не нужен	максимум скорости сети, сетевые утилиты; полноценно — на Linux
none	только loopback, наружу ни пакета	обработка файлов, недоверенный код
overlay	одна сеть поверх нескольких машин	кластер Docker Swarm; в Kubernetes — свои сети

! **Compose создаёт свою bridge-сеть сам:** имя сервиса = DNS-имя, порт — контейнерный: postgres://db:5432.

04

РАЗДЕЛ 04

Качество, отладка и среда как код

Маленький и безопасный образ, разбор поломок и devcontainers

Маленький образ и чистый контекст сборки

Большой образ дольше собирать, качать и хранить, и в нём больше уязвимостей. Берём лёгкую базу и не тащим лишнее в контекст сборки.

`python:3.12-slim`
≈ 145 МБ вместо ≈ 1,1 ГБ у полного python:3.12

тег, а не `latest`
версия зафиксирована — сборка воспроизводима

`--no-cache-dir`
pip не хранит кэш скачанных пакетов в образе

`.dockerignore`
что не отправлять демону при docker build

.dockerignore

```
.git
.venv
__pycache__/
*.pyc
.env
data/

# не уйдёт в контекст сборки
# и не попадёт в COPY . .
```

Multi-stage build: собираем в одном, отдаём другое

Собираем в «толстом» образе со всеми инструментами, а в финальный копируем только результат.

Dockerfile (multi-stage)

```
# стадия 1 — сборка
FROM python:3.12 AS build
COPY requirements.txt .
RUN pip install --prefix=/install \
    -r requirements.txt

# стадия 2 — то, что поедет на сервер
FROM python:3.12-slim
COPY --from=build /install /usr/local
COPY . /app
CMD ["python", "/app/app.py"]
```

build: компиляторы, заголовки, кэш — остаются тут

финальный образ: только результат

в образ попадает только последняя стадия — меньше места и меньше уязвимостей

Не root и без секретов внутри образа

По умолчанию процесс в контейнере работает от root. Понижаем права до обычного пользователя и не кладём секреты в образ.

непривилегированный пользователь

```
FROM python:3.12-slim
RUN useradd --create-home app
WORKDIR /home/app
COPY --chown=app:app . .
USER app
CMD ["python", "app.py"]

$ docker run app whoami
app
```

USER app
процесс без прав root внутри контейнера

секреты — мимо образа
не COPY .env; передать через -e при запуске

фиксируйте базу
тег или digest, а не latest

сканируйте
docker scout или trivy ищут уязвимости

Контейнер не работает: logs, exec, inspect

Контейнер упал или не отвечает. Четыре команды показывают, что происходит внутри.

терминал — ищем причину

```
$ docker ps -a
web   Exited (1)   5 seconds ago
$ docker logs web
ModuleNotFoundError: No module named 'fastapi'
# зависимость не попала в образ

$ docker inspect web --format '{{.State.ExitCode}}'
1
```

`docker logs`
stdout и stderr — смотрят первым делом

`exec -it web sh`
shell внутри работающего контейнера

`run -it --entrypoint sh`
упал сразу? запустить образ с shell

`inspect`
порты, тома, env, код выхода

Грабли новичка — и как их обойти

Эти ошибки встречаются в первых же Dockerfile. Каждая легко чинится, если знать, куда смотреть.

COPY . . слишком рано
Почему: любая правка кода ломает кэш — pip ставится заново

Порт не открывается
Почему: приложение слушает 127.0.0.1, нужно 0.0.0.0

Код изменил — в контейнере старый
Почему: забыли docker build или запустили старый тег

Данные пропали
Почему: не было тома — всё жило в слое контейнера

Огромный образ
Почему: полная база, а .git и .venv уехали в контекст

Секреты в образе
Почему: COPY .env остаётся в слоях, даже если потом удалить

Dev Containers: среда разработки как код

devcontainer.json описывает окружение разработки: образ, надстройки, команды при создании, расширения. VS Code и GitHub Codespaces поднимают по нему одинаковую среду у всех.

.devcontainer/devcontainer.json

```
{
  "build": {"dockerfile": "Dockerfile"},  # + uv
  "features": {".../docker-in-docker:2":
               {"moby": false}},
  "containerEnv": {"UV_PROJECT_ENVIRONMENT":
                   "/home/vscode/venv"},
  "runArgs": ["-p", "0.0.0.0:8000:8000"],
  "postCreateCommand": "bash .devcontainer/post-create.sh",
  "customizations": {"vscode": {"extensions":
      ["ms-python.python", "charliermarsh.ruff"]}}
}
```

`build`
свой образ среды: Python + uv

`features`
надстройки: свой Docker внутри

`postCreateCommand`
uv sync, ruff и pytest при создании

`customizations`
расширения и настройки VS Code

ИТОГИ ЛЕКЦИИ

Что важно унести

1. **Dockerfile — рецепт образа**
   шаги по порядку; порядок — это стратегия кэша
2. **Контейнер эфемерен**
   данные — в томах, порты — через -p, слушать 0.0.0.0
3. **Качество образа**
   slim-база, multi-stage, не root, без секретов
4. **Среда как код**
   devcontainer.json — одинаковое окружение у всех

Дальше в курсе — **Docker Compose**: приложение и база данных одной командой.


ВЫСШАЯ • ШКОЛА • ЭКОНОМИКИ •